Class 6: R Functions

Alexia (A17297003)

Table of contents

Background 1
A first function 1
Write a generate_dna() function 3
Write a `generate_protein()` function 5
Generate random protein sequences of length 6 to 13 5
Are our peptides "unique in nature"? 6
Connecting our findings to immunology 7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma seperated inputs inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either *required* or *optional*. The later have fall-back default values that wull be used if the user does not specify them.

```
add <- function(x, y=0) {
  x + y
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

> **Q1b**: For you second version, adapt your first function so it can take a single input vector or two inputs as before.

```
add <- function(x, y=0) {
  sum(x + y)
}
```

```
add (4, 7)
```

```
[1] 11
```

```
add ( c(4,7,10))
```

```
[1] 21
```

> **Q1c** On your own create a third version of your function that can add any number of inputs that the user provides

```
add <- function(x, y=0, z=0, d=0) {
  sum(x,y,z,d)
}
 add(4,7)
```

```
[1] 11
```

```
 add(c(1,2,3,-4))
```

```
[1] 2
```

We can explicitly set a **return** value output from a funcion (rather than the default last line) by using the `return()` function call.

```
add <- function(x, y=0, z=0){
  sum(x,y,z)
  cat("Is it break time yet?\n")
}

add(10,100)
```

```
Is it break time yet?
```

Write a generate_dna() function

A useful function here is the "base R" `sample()` function:

```
sample(1:5, size =4)
```

```
[1] 1 3 4 2
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G", and "T"...

```
sample(x=c("A", "C", "G", "T"), size=10, replace=TRUE)
```

```
 [1] "G" "A" "C" "C" "A" "C" "G" "C" "C" "G"
```

> **Q2a**: Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides.

```
generate_dna <- function(len){
  sample(x=c("A", "C", "G", "T"), size=len, replace=TRUE)
}
```

```
generate_dna()
```

```
[1] "T" "C" "A" "G"
```

```
#generate_dna(len=10)
```

Q2b Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a *single character string*

```
generate_dna <- function(len=10, single.element=TRUE){

  ans<-sample(x=c("A", "C", "G", "T"), size=len, replace=TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna()
```

```
[1] "CGAAACCATA"
```

```
generate_dna(single.element=FALSE)
```

```
 [1] "A" "A" "G" "G" "G" "G" "T" "C" "T" "G"
```

Q2c Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
generate_dna <- function(len=10, single.element=TRUE){

  ans<-sample(x=c("A", "C", "G", "T"), size=len, replace=TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }

  ##Format as FASTA with an ID line
  cat(paste(">length", len, "\n", sep=""))
  cat(ans)
  cat("\n")
  ##
  ##
}
```

```
generate_dna(21)
```

```
>length21
ACACCATTGTTAAGTACAATT
```

Write a `generate_protein()` function

> **Q3** Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user.

```
generate_protein <- function(len=7){
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S",
"T", "W", "Y", "V")
  ans <- sample(x=aa, size=len, replace=TRUE)
  paste(ans, collapse = "")
  }
```

```
generate_protein(7)
```

```
[1] "VHWHGAE"
```

Generate random protein sequences of length 6 to 13

> **Q4** Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call generate_protein() eight times by hand.

```
for(l in 6:13) {
  cat(">", l, "\n", sep="")
  cat( generate_protein(l), "\n")
}
```

```
>6
FWQTVG
>7
EKKEWII
>8
SCTYRYMR
```

```
>9
HYLVIGQWM
>10
HYAAEAIPWT
>11
NAHCNSNWIAE
>12
NTNHVGTICNGF
>13
QRFPMSDLECLAV
```

Are our peptides "unique in nature"?

Q5 : Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database

Length	Ide (%)	Cov (%)	Unique
6	100	100	No
7	100	100	No
8	87	100	Yes
9	100	77	Yes
10	77	100	Yes
11	100	64	Yes
12	44	100	Yes
13	57	100	Yes

Q5a At which sequence length do your randomly generated peptides start to look "unique in nature"?

My randomly generated peptides start to look "unique in nature" at sequence length 8.

Q5b Speculate why very short random peptides are almost always found in nr, while longer ones typically are not.

It is possible that very short random peptides are almost always found in nr because short peptides are more likely to be unique due to all of the combinations that can be generated compared to the limited diversity in longer random peptides.

Connecting our findings to immunology

> **Q6** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on the data from Q5 and the reasoning from Q5b I think that the minimum length is 8 as that is when the sequences start to come back as unique. It might be a bad design choice for the immune system to present very short peptides as they wouldn't be able to bind effectively to MHC molecules. Additionally, because of their vast diversity short peptides might resemble MHC molecules increasing the risk of autoimmunity developing.